# C# Programming for Indian Students — Chapter 5: Decision Making

Nov 2, 2026 · @Vikash

## 5.1 What you will learn

In Chapter 4 every line of a program ran, one after another. Real programs **choose**: a student passes or fails, a bill uses a different rate after 100 units, a wrong input gets a polite message instead of a crash. By the end of this chapter you will be able to:

- run a block of code only when a condition is true, with `if`;
- choose between two paths with `if` - `else` and the short form `?:`;
- choose between many paths with an `else if` ladder and with nested `if`;
- check the user's input with `int.TryParse` before using it;
- choose by value with the `switch` statement and the newer **switch expression**;
- build a Student Result Calculator that prints a report card with result and grade.

**How input programs are shown.** When a program asks the user to type something, the output shows the typed value after the prompt, as it appears on screen. Every program in this chapter was compiled and run on the .NET 10 SDK with the sample input shown.

**Remember:** all programs use top-level statements, as in Chapters 3 and 4. Each one is a list of lines you place in `Program.cs` of a new console project.

## 5.2 The if statement

**An `if` statement runs a block of code only when its condition is true.** The condition is any expression that gives a `bool`, such as `age >= 18` from Chapter 4.

```
if (condition)
{
    // runs only when condition is true
}
// the program continues here in both cases
```

The condition always sits inside round brackets. The statements to run sit inside curly braces `{ }`, called a **block**. When the condition is false, C# skips the whole block and continues after the closing brace.

**Program 5.1: The if statement**

Vikash Kumar

```
Console.Write("Enter your age: ");
int age = int.Parse(Console.ReadLine() ?? "");

if (age >= 18)
{
    Console.WriteLine("You are eligible to vote.");
}

Console.WriteLine("Thank you for checking.");
```

**Output** (two runs: the user typed `20`, then `16`)

```
Enter your age: 20
You are eligible to vote.
Thank you for checking.
```

```
Enter your age: 16
Thank you for checking.
```

**How it works.** With 20, `age >= 18` is `True`, so the message inside the block prints. With 16 the condition is `False`: the block is skipped and only the last line runs, because it sits outside the braces.

**Remember:** always write the braces, even for one statement. C# allows `if (age >= 18) Console.WriteLine("...");` without them, but a second line added later will *not* be part of the `if` (practice question 6 shows the trap).

## 5.3 if-else and the conditional operator

**`if` - `else` chooses between exactly two paths: one runs when the condition is true, the other when it is false.** Exactly one of the two blocks runs, never both and never neither.

```
if (condition)
{
    // runs when condition is true
}
else
{
    // runs when condition is false
}
```

**Program 5.2: if-else (even or odd)**

Vikash Kumar

```
Console.Write("Enter a number: ");
int number = int.Parse(Console.ReadLine() ?? "");

if (number % 2 == 0)
{
    Console.WriteLine($"{number} is even.");
}
else
{
    Console.WriteLine($"{number} is odd.");
}
```

**Output** (the user typed `47`)

```
Enter a number: 47
47 is odd.
```

**How it works.** `47 % 2` is 1 (Chapter 4), so `number % 2 == 0` is `False` and the `else` block runs. Note the two equals signs: `==` compares, `=` assigns.

## The conditional operator ?:

When both paths only produce a **value**, the conditional operator (also called the **ternary operator**, because it has three parts) does the job in one line:

```
condition ? valueIfTrue : valueIfFalse
```

**Program 5.3: The conditional operator ?:**

```
int marks = 72;
int attendance = 68;

string result = marks >= 40 ? "Pass" : "Fail";
string hallTicket = attendance >= 75 ? "Issued" : "Withheld (attendance below
75%)";

Console.WriteLine($"Marks      : {marks} -> {result}");
Console.WriteLine($"Attendance : {attendance}% -> Hall ticket {hallTicket}");

int a = 34;
int b = 81;
int larger = a > b ? a : b;
Console.WriteLine($"Larger of {a} and {b} is {larger}");
```

**Output**

Vikash Kumar

```
Marks      : 72 -> Pass
Attendance : 68% -> Hall ticket Withheld (attendance below 75%)
Larger of 34 and 81 is 81
```

**How it works.** `marks >= 40` is true, so `result` gets `"Pass"`. `attendance >= 75` is false, so `hallTicket` gets the second value. `a > b` is false, so `larger` gets `b`, which is 81. Both values after `?` and `:` must have the same type, or one must convert to the other (here both `string` or both `int`).

**Remember:** use `?:` to pick a value; use `if`-`else` when each path does work (prints, calculates, reads input). A long chain of `?:` inside `?:` is hard to read: use `if`-`else` or a switch expression (section 5.8) instead.

## 5.4 The else-if ladder

**An `else if` ladder chooses one path out of many: C# checks the conditions from top to bottom and runs the block of the first one that is true.** The remaining conditions are not checked at all. The final `else` is the "none of the above" path.

```
if (condition1)
{
    // runs when condition1 is true
}
else if (condition2)
{
    // runs when condition1 is false and condition2 is true
}
else
{
    // runs when every condition is false
}
```

**Example: an electricity bill with slabs.** Electricity boards in India charge more per unit as consumption grows, and each slab rate applies only to the units inside that slab. The rates below are example values, not a real tariff.

| Units | Rate per unit | For 250 units |
|---|---|---|
| First 100 | Rs. 3.00 | 100 × 3.00 = 300.00 |
| 101 to 200 | Rs. 4.50 | 100 × 4.50 = 450.00 |
| 201 to 300 | Rs. 6.00 | 50 × 6.00 = 300.00 |

Vikash Kumar

| Units | Rate per unit | For 250 units |
|---|---|---|
| Above 300 | Rs. 7.50 | none |

### Program 5.4: else-if ladder (electricity bill)

```
Console.Write("Units consumed: ");
int units = int.Parse(Console.ReadLine() ?? "");

decimal bill;

if (units <= 100)
{
    bill = units * 3.00m;
}
else if (units <= 200)
{
    bill = 100 * 3.00m + (units - 100) * 4.50m;
}
else if (units <= 300)
{
    bill = 100 * 3.00m + 100 * 4.50m + (units - 200) * 6.00m;
}
else
{
    bill = 100 * 3.00m + 100 * 4.50m + 100 * 6.00m + (units - 300) * 7.50m;
}

Console.WriteLine($"Bill amount   : Rs. {bill:F2}");
```

**Output** (the user typed `250`)

```
Units consumed: 250
Bill amount   : Rs. 1050.00
```

**How it works.** 250 is not `<= 100` and not `<= 200`, but it is `<= 300`, so the third block runs: 300 + 450 + (250 − 200) × 6 = 1050. Because the ladder stops at the first true condition, the third test does not need to say `units > 200 && units <= 300`: reaching it already means `units > 200`.

`bill` is declared without a value. The compiler accepts `Console.WriteLine(... bill ...)` only because **every** path, including the final `else`, gives `bill` a value. Remove the `else` and you get error CS0165 (section 5.9).

**Remember:** in a ladder, order matters. Put the narrow or the smallest range first. If `units <= 300` came first, 50 units would be charged at the wrong rate.

Vikash Kumar

## 5.5 Nested if

**A nested `if` is an `if` inside the block of another `if`: the inner question is asked only when the outer answer is already known.** Use it when the second check makes sense only after the first one passes.

**Example: college admission.** A college first checks the Class 12 percentage. Only for students who clear it does it look at whether they studied Maths. The rules here are example rules, not those of a real university.

**Program 5.5: Nested if (admission check)**

```
Console.Write("Percentage in Class 12 : ");
double percentage = double.Parse(Console.ReadLine() ?? "");

Console.Write("Studied Maths (y/n)    : ");
string answer = Console.ReadLine() ?? "";
bool hasMaths = answer == "y" || answer == "Y";

if (percentage >= 50)
{
    if (hasMaths)
    {
        Console.WriteLine("Eligible for B.Tech and BCA.");
    }
    else
    {
        Console.WriteLine("Eligible for BCA. B.Tech needs Maths in Class
12.");
    }
}
else
{
    Console.WriteLine("Not eligible: a minimum of 50% is required.");
}
```

**Output** (the user typed `72.4` and `n`)

```
Percentage in Class 12 : 72.4
Studied Maths (y/n)    : n
Eligible for BCA. B.Tech needs Maths in Class 12.
```

**How it works.** 72.4 is at least 50, so the outer block runs and asks the inner question. `hasMaths` is `False` because the user typed `n`, so the inner `else` prints. A student with 45% never reaches the inner `if`: the outer `else` prints "Not eligible".

Vikash Kumar

`hasMaths` stores the answer in a `bool` with a readable name. The condition `if (hasMaths)` then reads like English; `if (hasMaths == true)` works too but adds nothing.

**Nested if or &&?** The same rules can be written as a ladder with `&&`:

```
if (percentage >= 50 && hasMaths) { ... }
else if (percentage >= 50) { ... }
else { ... }
```

The ladder tests `percentage >= 50` twice. Nesting tests it once and shows that the Maths question belongs to students who passed the first check.

**Remember:** indent each inner block by four more spaces (Visual Studio does it for you). More than two or three levels of nesting is hard to follow: that is a sign to use `&&` or, in a later chapter, a method.

## 5.6 Checking input with TryParse and if

**`int.TryParse` returns `True` or `False`, and `if` turns that answer into a friendly message instead of a crash.** In Chapter 4, typing `abc` for `int.Parse` stopped the program with a `FormatException`. Now we can check first.

A good input check asks two questions, in this order:

1. Is the text a number at all? (`int.TryParse`)
2. Is the number in the allowed range? (relational and logical operators)

**Program 5.6: Checking input before using it**

```
Console.Write("Enter marks (0 to 100): ");
string text = Console.ReadLine() ?? "";

if (!int.TryParse(text, out int marks))
{
    Console.WriteLine($"'{text}' is not a whole number. Please run the
program again.");
}
else if (marks < 0 || marks > 100)
{
    Console.WriteLine($"{marks} is outside the range 0 to 100.");
}
else
{
    Console.WriteLine($"Marks accepted: {marks}");
}
```

Vikash Kumar

**Output** (three runs: the user typed `abc`, `120` and `86`)

```
Enter marks (0 to 100): abc
'abc' is not a whole number. Please run the program again.
```

```
Enter marks (0 to 100): 120
120 is outside the range 0 to 100.
```

```
Enter marks (0 to 100): 86
Marks accepted: 86
```

**How it works.** `int.TryParse(text, out int marks)` tries the conversion. It returns `True` and stores the number in `marks`, or returns `False` and stores 0. The `!` reverses the answer, so the first block runs only when conversion **failed**. The `else if` is reached only with a real number, and `marks < 0 || marks > 100` catches values out of range. Only good input reaches the final `else`.

**Why this order?** Checking the range first would use `marks` before we know it holds a real number: for `abc`, `marks` is 0 and would wrongly pass a test such as `marks >= 0`.

**Remember:** never trust input. A program that real users run should check every number with `TryParse` and `if`. In Chapter 6 you will use a loop to ask again until the input is valid, instead of asking the user to run the program again.

## 5.7 The switch statement

**A `switch` statement compares one value with a list of fixed values and runs the matching `case`.** It is neater than a long `else if` ladder when every test has the form `variable == something`.

```
switch (value)
{
    case constant1:
        // statements
        break;
    case constant2:
        // statements
        break;
    default:
        // runs when no case matches
        break;
}
```

The rules:

Virash Kumar

- Each `case` label is a constant: a number, a character such as `'A'` or a string such as `"+"`.
- Each section ends with `break;`, which jumps out of the `switch`. C# does **not** allow one section to fall into the next (error CS0163, section 5.9).
- Several labels may share one section: write them one under another with no statements between.
- `default` runs when no label matches. It is optional, but always write it.

**Example: month to season.** India has four broad seasons: summer (March to May), monsoon (June to September), post-monsoon (October and November) and winter (December to February).

**Program 5.7: The switch statement (month to season)**

```
Console.Write("Enter month number (1-12): ");
int month = int.Parse(Console.ReadLine() ?? "");

string season;

switch (month)
{
    case 3:
    case 4:
    case 5:
        season = "Summer";
        break;
    case 6:
    case 7:
    case 8:
    case 9:
        season = "Monsoon";
        break;
    case 10:
    case 11:
        season = "Post-monsoon";
        break;
    case 12:
    case 1:
    case 2:
        season = "Winter";
        break;
    default:
        season = "Invalid month";
```

Vikash Kumar

```
        break;
}

Console.WriteLine($"Season: {season}");
```

**Output** (the user typed `8`)

```
Enter month number (1-12): 8
Season: Monsoon
```

**How it works.** C# jumps straight to `case 8:`, which shares its section with 6, 7 and 9, sets `season` to `"Monsoon"` and leaves at `break`. With 13, no label matches and `default` sets `"Invalid month"`. Because `default` also assigns `season`, the compiler knows it always has a value before the last line.

**switch or else-if?** Use `switch` for exact values (menu choices, day or month numbers, grade letters, operators). Use an `else if` ladder for ranges such as `marks >= 75`, or wait for the switch expression in the next section, which handles ranges too.

**Remember:** the labels in one `switch` must all be different. Two `case 1:` labels give error CS0152.

## 5.8 Switch expressions and patterns

**A switch expression picks one value out of many in a single expression, and its patterns can test ranges, not only exact values.** It was added in C# 8 and is the modern way to write "choose a value" code.

```
result = value switch
{
    pattern1 => result1,
    pattern2 => result2,
    _ => resultWhenNothingMatches
};
```

Compared with the `switch` statement: the value comes **before** the word `switch`, each line is an **arm** of the form `pattern => result`, arms are separated by commas, there is no `case` and no `break`, and `_` (the **discard**) plays the role of `default`. The whole thing produces a value, so it ends with a semicolon.

Patterns you can use with numbers:

Vikash Kumar

| Pattern | Matches | Example |
|---|---|---|
| Constant | one exact value | `7` |
| Relational | a comparison with `<` `<=` `>` `>=` | `>= 60` |
| `or` | either pattern | `6 or 7` |
| `and` | both patterns | `>= 1 and <= 5` |
| `not` | the opposite | `not 0` |
| Discard `_` | anything | `_` |

### Program 5.8: Switch expressions and patterns

```
Console.Write("Enter month number (1-12): ");
int month = int.Parse(Console.ReadLine() ?? "");

string season = month switch
{
    3 or 4 or 5 => "Summer",
    >= 6 and <= 9 => "Monsoon",
    10 or 11 => "Post-monsoon",
    12 or 1 or 2 => "Winter",
    _ => "Invalid month"
};
Console.WriteLine($"Season: {season}");

Console.Write("Enter passenger age: ");
int age = int.Parse(Console.ReadLine() ?? "");

decimal fare = 500m;
decimal payable = age switch
{
    < 5 => 0m,
    < 12 => fare / 2,
    >= 60 => fare * 0.6m,
    _ => fare
};
Console.WriteLine($"Fare: Rs. {fare:F2}, payable: Rs. {payable:F2}");
```

**Output** (the user typed `8` and `67`)

Vikash Kumar

```
Enter month number (1-12): 8
Season: Monsoon
Enter passenger age: 67
Fare: Rs. 500.00, payable: Rs. 300.00
```

**How it works.** The first switch expression is Program 5.7 in five lines. `>= 6 and <= 9` matches 6 to 9 in one arm. The second picks a fare with **relational patterns** (example concessions: free under 5, half fare under 12, 40% off from 60). Arms are tried from top to bottom, like an `else if` ladder, so `< 12` only sees ages 5 and above: an age of 3 has already matched `< 5`. 67 fails `< 5` and `< 12`, matches `>= 60`, and pays 500 × 0.6 = 300.

**Two compiler checks help you here.** If an arm can never be reached because an earlier arm already covers it, the build fails with CS8510. If no arm matches some possible value and there is no `_`, you get warning CS8509; at run time an unmatched value stops the program with a `SwitchExpressionException`. Section 5.9 shows both messages.

**Remember:** finish a switch expression with a `_` arm unless every possible value is already covered. Use a switch **expression** to pick a value; use a switch **statement** when each case must do several things, such as printing and reading input.

## 5.9 Dry run and common errors

**Dry run: decisions one step at a time.** Follow this program and write down, for each `if`, whether its condition is true or false, and the value of each variable after every line.

**Program 5.9: Dry run**

```
int marks = 37;
int bonus = 0;
string remark;

if (marks >= 35 && marks < 40)
{
    bonus = 40 - marks;
}

marks += bonus;

if (marks >= 60)
{
    remark = "First division";
}
else if (marks >= 45)
{
    remark = "Second division";
```

Vikash Kumar

```
}
else if (marks >= 40)
{
    remark = "Pass";
}
else
{
    remark = "Fail";
}

Console.WriteLine($"marks = {marks}, bonus = {bonus}, remark = {remark}");
```

| Step | Statement | Working | marks | bonus | remark |
|---|---|---|---|---|---|
| 1 | `int marks = 37;` | | 37 | - | - |
| 2 | `int bonus = 0;` | | 37 | 0 | - |
| 3 | `string remark;` | declared, no value yet | 37 | 0 | - |
| 4 | `if (marks >= 35 && marks < 40)` | True && True: enter the block | 37 | 0 | - |
| 5 | `bonus = 40 - marks;` | 40 − 37 = 3 grace marks | 37 | 3 | - |
| 6 | `marks += bonus;` | 37 + 3 | 40 | 3 | - |
| 7 | `if (marks >= 60)` | False: go to the next test | 40 | 3 | - |
| 8 | `else if (marks >= 45)` | False: go to the next test | 40 | 3 | - |
| 9 | `else if (marks >= 40)` | True: run this block, skip the rest | 40 | 3 | Pass |

### Output

```
marks = 40, bonus = 3, remark = Pass
```

Try it with `marks = 30`: step 4 is False, `bonus` stays 0, and the last `else` gives "Fail".

**Common errors.** Each message below is the real message from the .NET 10 compiler.

| Mistake | Wrong code | Message | Fix |
|---|---|---|---|
| `=` instead of `==` in a condition | `if (marks = 40)` | error CS0029: Cannot implicitly convert type 'int' to 'bool' | `if (marks == 40)` |
| Semicolon after `if (...)` | `if (marks >= 40);` | warning CS0642: Possible mistaken empty statement | Remove the `;` after `)` |

Vikash Kumar

| Mistake | Wrong code | Message | Fix |
|---|---|---|---|
| Missing `break` in a `switch` section | `case 1: Console.WriteLine("Tea");` then `case 2:` | error CS0163: Control cannot fall through from one case label ('case 1:') to another | End every section with `break;` |
| Variable not given a value on every path | `string grade;` set only in `if` and `else if`, then printed | error CS0165: Use of unassigned local variable 'grade' | Add a final `else` that assigns it |
| Comparing only one side | `if (answer == "y" \|\| "Y")` | error CS0019: Operator '\|\|' cannot be applied to operands of type 'bool' and 'string' | `if (answer == "y" \|\| answer == "Y")` |
| Same label twice | two `case 1:` labels | error CS0152: The switch statement contains multiple cases with the label value '1' | Make every label different |
| Arm hidden by an earlier arm | `>= 40 => "Pass", >= 60 => "First"` | error CS8510: The pattern is unreachable. It has already been handled by a previous arm of the switch expression or it is impossible to match. | Put the narrower range first: `>= 60` before `>= 40` |

Vikash Kumar

| Mistake | Wrong code | Message | Fix |
|---|---|---|---|
| No `_` arm | `day switch { 6 => "Saturday", 7 => "Sunday" }` | warning CS8509: The switch expression does not handle all possible values of its input type (it is not exhaustive). For example, the pattern '0' is not covered. | Add `_ => "..."` as the last arm |

**Two warnings that become wrong behaviour.** A warning does not stop the build, so the program still runs. With `if (marks >= 40);` and `marks = 25`, the program prints "Pass": the `;` is an empty statement that belongs to the `if`, and the block after it always runs. With the CS8509 code and `day = 9`, the program stops with:

```
Unhandled exception.
System.Runtime.CompilerServices.SwitchExpressionException: Non-exhaustive
switch expression failed to match its input.
```

**Logic errors** give no message at all. Examples from this chapter: a ladder in the wrong order (`marks >= 40` before `marks >= 75`, practice question 7), `marks > 40` when the rule says 40 or more, and `||` where the rule needs both conditions (`&&`).

**Remember:** read warnings, do not ignore them. Visual Studio lists them in the Error List window (View > Error List).

## 5.10 Mini project: Student Result Calculator

**Problem.** A school wants a program that prints a report card. The teacher types the student's name and marks in five subjects. The program checks the input, works out the total and percentage, counts the subjects failed, and prints the result and a grade.

**Rules** (example rules for this project, not those of a real board)

| Item | Rule |
|---|---|
| Valid marks | Whole numbers from 0 to 100 in every subject |
| Pass mark | 33 in each subject |

Vikash Kumar

| Item | Rule |
|---|---|
| Result | PASS if no subject is failed, SUPPLEMENTARY if 1 or 2 are failed, FAIL if 3 or more are failed |
| Grade (PASS only) | A+ for 90% and above, A for 75%, B for 60%, C for 45%, D below 45% |
| Percentage | total × 100 / 500 |

### Program 5.10: Student Result Calculator

```
const int PassMark = 33;
const int MaxTotal = 500;

Console.Write("Student name : ");
string name = Console.ReadLine() ?? "";

Console.Write("English      : ");
bool ok1 = int.TryParse(Console.ReadLine(), out int english);
Console.Write("Hindi        : ");
bool ok2 = int.TryParse(Console.ReadLine(), out int hindi);
Console.Write("Maths        : ");
bool ok3 = int.TryParse(Console.ReadLine(), out int maths);
Console.Write("Science      : ");
bool ok4 = int.TryParse(Console.ReadLine(), out int science);
Console.Write("Computer     : ");
bool ok5 = int.TryParse(Console.ReadLine(), out int computer);

if (!(ok1 && ok2 && ok3 && ok4 && ok5))
{
    Console.WriteLine("Please enter whole numbers only.");
}
else if (english < 0 || english > 100 || hindi < 0 || hindi > 100
         || maths < 0 || maths > 100 || science < 0 || science > 100
         || computer < 0 || computer > 100)
{
    Console.WriteLine("Marks must be between 0 and 100.");
}
else
{
    int total = english + hindi + maths + science + computer;
    double percentage = total * 100.0 / MaxTotal;

    int failedSubjects = 0;
    failedSubjects += english < PassMark ? 1 : 0;
    failedSubjects += hindi < PassMark ? 1 : 0;
```

Vikash Kumar

```
    failedSubjects += maths < PassMark ? 1 : 0;
    failedSubjects += science < PassMark ? 1 : 0;
    failedSubjects += computer < PassMark ? 1 : 0;

    string result = failedSubjects switch
    {
        0 => "PASS",
        1 or 2 => "SUPPLEMENTARY",
        _ => "FAIL"
    };

    string grade;
    if (result == "PASS")
    {
        grade = percentage switch
        {
            >= 90 => "A+",
            >= 75 => "A",
            >= 60 => "B",
            >= 45 => "C",
            _ => "D"
        };
    }
    else
    {
        grade = "-";
    }

    Console.WriteLine();
    Console.WriteLine("==================================");
    Console.WriteLine("       REPORT CARD");
    Console.WriteLine("==================================");
    Console.WriteLine($"Name       : {name}");
    Console.WriteLine($"Total      : {total} / {MaxTotal}");
    Console.WriteLine($"Percentage : {percentage:F2}%");
    Console.WriteLine($"Failed in  : {failedSubjects} subject(s)");
    Console.WriteLine($"Result     : {result}");
    Console.WriteLine($"Grade      : {grade}");
    Console.WriteLine("==================================");
}
```

Vikash Kumar

**Output** (first run: the user typed `Ananya Iyer`, `82`, `76`, `91`, `68`, `95`)

```
Student name : Ananya Iyer
English      : 82
Hindi        : 76
Maths        : 91
Science      : 68
Computer     : 95

==================================
        REPORT CARD
==================================
Name       : Ananya Iyer
Total      : 412 / 500
Percentage : 82.40%
Failed in  : 0 subject(s)
Result     : PASS
Grade      : A
==================================
```

**Output** (second run: the user typed `Rohit Verma`, `45`, `28`, `61`, `30`, `72`)

```
Student name : Rohit Verma
English      : 45
Hindi        : 28
Maths        : 61
Science      : 30
Computer     : 72

==================================
        REPORT CARD
==================================
Name       : Rohit Verma
Total      : 236 / 500
Percentage : 47.20%
Failed in  : 2 subject(s)
Result     : SUPPLEMENTARY
Grade      : -
==================================
```

**How it works.**

1. **Read and convert.** Each `int.TryParse` stores its success in `ok1` to `ok5`. `TryParse` accepts the possibly-null result of `ReadLine` directly, so `?? ""` is not needed here.
2. **Check.** The first test catches any non-number; the `else if` catches any mark outside 0 to 100. Only clean input reaches the calculation, exactly as in Program 5.6.

Vikash Kumar

3. **Calculate.** `total * 100.0 / MaxTotal` uses `100.0` so the division keeps its fraction (Chapter 4): 412 × 100.0 / 500 = 82.4.
4. **Count failures.** Each `?:` adds 1 when that subject is below 33 and 0 otherwise. Rohit has 28 in Hindi and 30 in Science, so `failedSubjects` is 2.
5. **Decide.** One switch expression turns the count into a result. A second one turns the percentage into a grade, with the highest range first, as section 5.9 requires. Only a PASS gets a grade.

**Decisions used and why**

| Need | Tool | Reason |
|---|---|---|
| Reject bad input | `if` / `else if` / `else` | Three paths that each do different work |
| Count one subject | `?:` | Picks the value 1 or 0 |
| Result from a count | switch expression | Exact values 0, 1 or 2, and the rest |
| Grade from a percentage | switch expression with relational patterns | Ranges, highest first |

**Try it yourself**

1. Run the program with marks `95, 92, 99, 90, 88` and check that the grade is A+.
2. Type `ninety` for one subject, then `105`. Read the two messages.
3. Add a line `Division : First / Second / Third` for PASS results, using an `else if` ladder (60% and above First, 45% Second, otherwise Third).
4. Change the pass mark to 40 in one place only. Why does the program still work everywhere?

## 5.11 Practice questions

Try each question on your own first. The answers follow the list.

### Section A: Short answer

1. What is the difference between `if` and `if-else`?
2. In an `else if` ladder, how many blocks can run? What happens to the conditions after the first true one?
3. Why does every section of a `switch` statement end with `break`?

Vikash Kumar

4. Write the conditional operator that stores `"Adult"` or `"Minor"` in `category` for a given `age`.
5. Why should a program check input with `int.TryParse` before using it?

## Section B: Predict the output

6. What does this program print?

```
int x = 15;

if (x > 10)
    Console.WriteLine("A");
    Console.WriteLine("B");

if (x > 20)
    Console.WriteLine("C");
    Console.WriteLine("D");
```

7. What does this program print?

```
int marks = 85;

if (marks >= 40)
{
    Console.WriteLine("Pass");
}
else if (marks >= 75)
{
    Console.WriteLine("Distinction");
}
else
{
    Console.WriteLine("Fail");
}

string kind = marks % 2 == 0 ? "even" : "odd";
Console.WriteLine(kind);
```

8. What does this program print?

```
char grade = 'B';

switch (grade)
{
    case 'A':
        Console.WriteLine("Excellent");
```

Vikash Kumar

```
        break;
    case 'B':
    case 'C':
        Console.WriteLine("Good");
        break;
    default:
        Console.WriteLine("Keep trying");
        break;
}

int day = 9;
string type = day switch
{
    6 or 7 => "Weekend",
    >= 1 and <= 5 => "Weekday",
    _ => "Invalid"
};
Console.WriteLine(type);
```

## Section C: Find the error

9. This prints "You passed" even for 25 marks. Why?

```
if (marks >= 40);
{
    Console.WriteLine("You passed");
}
```

10. This does not compile. Why?

```
switch (choice)
{
    case 1:
        Console.WriteLine("Tea");
    case 2:
        Console.WriteLine("Coffee");
        break;
}
```

## Section D: Write the program

11. Read three whole numbers and print the largest.
12. Read a year and print whether it is a leap year. (A year is a leap year if it is divisible by 4 but not by 100, or if it is divisible by 400.)

Vikash Kumar

13. Read two numbers and an operator (`+`, `-`, `*` or `/`) and print the result using `switch`. Print a message instead of dividing by zero.
14. Read the three sides of a triangle. Print whether they can form a triangle, and if so whether it is equilateral, isosceles or scalene. (Each side must be positive and the sum of any two sides must be greater than the third.)

## Answers

1. `if` runs a block only when the condition is true and does nothing otherwise. `if` - `else` always runs exactly one of two blocks.
2. At most one block runs: the first whose condition is true, or the final `else` if none is. The conditions after the first true one are not checked.
3. C# does not let control fall from one section into the next. `break` ends the section and leaves the `switch`; without it the build fails with CS0163.
4. `string category = age >= 18 ? "Adult" : "Minor";`
5. `int.Parse` stops the program with a `FormatException` when the user types text such as `abc`. `int.TryParse` returns `False` instead, so an `if` can print a message and the program keeps control.
6. Output:

   ```
   A
   B
   D
   ```

   Without braces, an `if` controls only the **next one** statement. `Console.WriteLine("B")` and `Console.WriteLine("D")` are outside the `if` statements, whatever the indentation suggests, so they always run. `C` is skipped because 15 is not greater than 20.
7. Output:

   ```
   Pass
   odd
   ```

   85 matches the first test, `marks >= 40`, so the ladder stops there and "Distinction" is never reached. The ladder is in the wrong order: put `marks >= 75` first. 85 % 2 is 1, so `kind` is "odd".
8. Output:

   ```
   Good
   Invalid
   ```

Vikash Kumar

`'B'` shares its section with `'C'`. 9 is not 6 or 7 and not between 1 and 5, so the `_` arm gives "Invalid".

9. The `;` right after `if (marks >= 40)` is an empty statement, and that empty statement is all the `if` controls. The block in braces is an ordinary block that always runs. The compiler warns: `warning CS0642: Possible mistaken empty statement`. Remove the `;`.
10. `case 1:` has no `break`, so control would fall into `case 2:`: `error CS0163: Control cannot fall through from one case label ('case 1:') to another`. Add `break;` after `Console.WriteLine("Tea");`.
11. One possible answer:

```
Console.Write("First number  : ");
int a = int.Parse(Console.ReadLine() ?? "");
Console.Write("Second number : ");
int b = int.Parse(Console.ReadLine() ?? "");
Console.Write("Third number  : ");
int c = int.Parse(Console.ReadLine() ?? "");

int largest;

if (a >= b && a >= c)
{
    largest = a;
}
else if (b >= c)
{
    largest = b;
}
else
{
    largest = c;
}

Console.WriteLine($"Largest       : {largest}");
```

```
First number  : 45
Second number : 92
Third number  : 67
Largest       : 92
```

The `else if` does not need to compare `b` with `a` again: reaching it means `a` is not the largest.

12. One possible answer:

Vikash Kumar

```
Console.Write("Enter a year: ");
int year = int.Parse(Console.ReadLine() ?? "");

if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
{
    Console.WriteLine($"{year} is a leap year.");
}
else
{
    Console.WriteLine($"{year} is not a leap year.");
}
```

```
Enter a year: 2024
2024 is a leap year.
```

```
Enter a year: 1900
1900 is not a leap year.
```

1900 is divisible by 4 and by 100 but not by 400, so it is not a leap year.

13. One possible answer:

```
Console.Write("First number     : ");
double first = double.Parse(Console.ReadLine() ?? "");
Console.Write("Operator (+-*/)  : ");
string op = Console.ReadLine() ?? "";
Console.Write("Second number    : ");
double second = double.Parse(Console.ReadLine() ?? "");

switch (op)
{
    case "+":
        Console.WriteLine($"Result          : {first + second}");
        break;
    case "-":
        Console.WriteLine($"Result          : {first - second}");
        break;
    case "*":
        Console.WriteLine($"Result          : {first * second}");
        break;
    case "/":
        if (second == 0)
        {
            Console.WriteLine("Cannot divide by zero.");
        }
        else
```

Vikash Kumar

```
        {
            Console.WriteLine($"Result           : {first / second}");
        }
        break;
    default:
        Console.WriteLine($"Unknown operator '{op}'.");
        break;
}
```

```
First number     : 12
Operator (+-*/)  : *
Second number    : 5
Result           : 60
```

An `if` - `else` sits inside the `"/"` section: a section can hold any statements before its `break`.

14. One possible answer:

```
Console.Write("Side 1: ");
int s1 = int.Parse(Console.ReadLine() ?? "");
Console.Write("Side 2: ");
int s2 = int.Parse(Console.ReadLine() ?? "");
Console.Write("Side 3: ");
int s3 = int.Parse(Console.ReadLine() ?? "");

if (s1 <= 0 || s2 <= 0 || s3 <= 0
    || s1 + s2 <= s3 || s1 + s3 <= s2 || s2 + s3 <= s1)
{
    Console.WriteLine("These sides cannot form a triangle.");
}
else if (s1 == s2 && s2 == s3)
{
    Console.WriteLine("Equilateral triangle");
}
else if (s1 == s2 || s2 == s3 || s1 == s3)
{
    Console.WriteLine("Isosceles triangle");
}
else
{
    Console.WriteLine("Scalene triangle");
}
```

Vikash Kumar

```
Side 1: 5
Side 2: 5
Side 3: 8
Isosceles triangle
```

The invalid case is tested first, so the later tests only ever see a real triangle. Equilateral is tested before isosceles because an equilateral triangle also has two equal sides.

## 5.12 MCQs and viva questions

**Multiple-choice questions** (answers are listed after the questions)

1. The condition of an `if` must give a value of type:
   - (a) `int`
   - (b) `string`
   - (c) `bool`
   - (d) any type
2. With `int x = 5;`, what does `if (x > 3) { Console.WriteLine("Yes"); } else { Console.WriteLine("No"); }` print?
   - (a) Yes
   - (b) No
   - (c) Yes and No
   - (d) nothing
3. In an `else if` ladder, the final `else` runs when:
   - (a) the first condition is true
   - (b) every condition is false
   - (c) every condition is true
   - (d) always
4. What is the value of `10 > 7 ? "big" : "small"`?
   - (a) "small"
   - (b) True
   - (c) "big"
   - (d) an error
5. Which keyword ends a section of a `switch` statement?
   - (a) `stop`
   - (b) `break`
   - (c) `exit`

Vikash Kumar

   - (d) `end`
6. Which part of a `switch` statement runs when no `case` matches?
   - (a) `else`
   - (b) `default`
   - (c) `_`
   - (d) `otherwise`
7. In a switch expression, `_` means:
   - (a) zero
   - (b) an empty string
   - (c) any value not matched by an earlier arm
   - (d) an error
8. Which pattern matches the numbers 6 to 9 in a switch expression?
   - (a) `6 to 9`
   - (b) `>= 6 and <= 9`
   - (c) `6..9`
   - (d) `>= 6 or <= 9`
9. `if (marks = 40)` gives:
   - (a) error CS0029
   - (b) True when marks is 40
   - (c) a run-time exception
   - (d) no error
10. `int.TryParse("abc", out int n)` returns:
   - (a) True, and n is 0
   - (b) False, and n is 0
   - (c) a `FormatException`
   - (d) False, and n is "abc"

**MCQ answers:** 1-c, 2-a, 3-b, 4-c, 5-b, 6-b, 7-c, 8-b, 9-a, 10-b.

**Viva questions** (short oral questions with model answers)

1. **What is a decision-making statement?** A statement that chooses which code runs based on a condition: `if`, `if-else`, `else if`, `switch` and the switch expression.
2. **What is a block?** One or more statements inside `{ }`, treated as one unit by an `if` or `else`.
3. **What is a nested if?** An `if` written inside the block of another `if`, so the inner test runs only when the outer one is true.

Vikash Kumar

4. **Why is the ternary operator called ternary?** Because it takes three operands: the condition, the value if true and the value if false.
5. **Can a C# switch fall through like in C?** No. Each non-empty section must end with `break` (or another jump such as `return`), otherwise the build fails with CS0163. Several labels may share one section.
6. **Switch statement vs switch expression?** A statement runs code for each case; an expression produces a value and supports patterns such as `>= 60` and `6 or 7`.
7. **Why test the narrower range first?** Because a ladder or switch expression stops at the first match. `>= 40` before `>= 75` would catch 85 as a simple pass; in a switch expression the compiler reports CS8510.

## 5.13 Summary and what comes next

- `if (condition) { ... }` runs a block only when the `bool` condition is true. Always write the braces.
- `if` - `else` runs exactly one of two blocks. `condition ? a : b` picks one of two **values** in one line.
- An `else if` ladder checks conditions from top to bottom and runs only the first true one, so put the narrowest or highest range first and end with `else`.
- A nested `if` asks a second question only when the first answer is true; `&&` is often a flatter alternative.
- Check input in two steps: `int.TryParse` for "is it a number?", then a range test with `||` or `&&`.
- A `switch` statement matches exact values. Every section ends with `break`, several labels may share a section, and `default` handles everything else.
- A switch expression (`value switch { pattern => result, ... }`) produces a value and supports patterns such as `>= 60`, `6 or 7` and `>= 1 and <= 5`. End it with `_`.
- The compiler catches many decision mistakes: CS0029 (`=` for `==`), CS0163 (missing `break`), CS0165 (variable not set on every path), CS0152 (duplicate label), CS8510 (unreachable arm). Warnings CS0642 and CS8509 still let the program build, so read them.

**Next chapter:** Chapter 6, Loops, repeats code. You will write `for`, `while`, `do` - `while` and `foreach`, ask again until the input is valid instead of asking the user to run the program again, print multiplication tables and star patterns, and build a number-guessing game.

Vikash Kumar